



System Design

Table of contents

1. Domain-Driven Design (DDD)

[What is the idea?](#)

[Example from your system](#)

[Benefits](#)

[Downsides](#)

2. Hexagonal Architecture / Ports & Adapters

2.1 Ports & Adapters

🔴 [Where it appears](#)

[What does it mean?](#)

[Example](#)

[Domain Port:](#)

[Adapter:](#)

[Benefits](#)

[Downsides](#)

3. Inversion of Control (IoC)

🔴 [Where IoC appears in your system](#)

[Benefit Example](#)

[Benefits](#)

[Downsides](#)

4. Pipes & Filters (Pipeline Design)

[Where it appears](#)

[What does it mean?](#)

[Example from your system](#)

[Benefits](#)

[Downsides](#)

[Architecture Diagram](#)

1. Domain-Driven Design (DDD)

“Focus your code around the *business logic*, not the framework.”

In your architecture diagram, the **Core Domain** is the center:

```
1 Core-Domain
2   |— Domain Models
3   |— Domain Services
4   |— Ports (abstractions)
```

</> SQL

This lives in `backend/domain/`.

What is the idea?

DDD says:

- The **important part** of your app is the *business rules*, not APIs, not databases.
- So you put your “real logic” inside domain models + services.
- Everything else is just a *detail*.

Example from your system

Domain says:

“To extract text, I need something that behaves like `DocumentExtractionPort`.”

It doesn't know:

- ✗ how Docling works
- ✗ HTTP requests
- ✗ file storage
- ✗ chunk sizes

The domain only knows **what** must be done, not **how**.

Benefits

- **Very reusable** — the same domain can work with Docling, Tika, ML, etc.
- **Easy to modify** — replace Docling without touching business logic.
- **Keeps code clean** — business logic stays separated from frameworks.

Downsides

- Harder to understand at first for beginners.
- Requires more files → seems more complex.
- Needs discipline: if developers break boundaries, the architecture collapses.

2. Hexagonal Architecture / Ports & Adapters

“Business logic never depends on technical implementation.”

This is where **Inversion of Control (IoC)** comes in.

2.1 Ports & Adapters

📌 Where it appears

Your diagram clearly separates:

- ```
1 Ports (core-domain)
2 Adapters (infrastructure)
```

</> SQL

Examples:

- **Port:** `DocumentExtractionPort` (interface)
- **Adapter:** `DoclingExtractorAdapter` (actual Docling HTTP calls)

## What does it mean?

- **Ports** = interfaces defined in the domain
- **Adapters** = classes that implement the port using real technology

Domain → defines *what* it needs

Adapter → provides *how* it's done

## Example

**Domain Port:**

```
1 class DocumentExtractionPort(Protocol):
2 async def extract(self, file_bytes, filename): ...
```

</> Python

**Adapter:**

```
1 class DoclingExtractorAdapter(DocumentExtractionPort):
2 async def extract(...):
3 return docling_api_request(...)
4
```

</> Python

If tomorrow you replace Docling with Tika, only the adapter changes — domain stays untouched.

## Benefits

- **Pluggable architecture**

Swap Docling for another extractor without changing your pipelines.

- **Perfect for microservices**

If tomorrow you move extraction to another service, domain doesn't care.

- **Testability**

Mock the port → test the domain without running Docling or a database.

- **Loose coupling**

Domain doesn't depend on slow APIs / fragile infrastructure.

## Downsides

- More files (lots of abstractions).
- More complex for beginners ("why 3 files for one feature??").

## 3. Inversion of Control (IoC)

“Low-level modules plug into high-level modules, not the other way around.”

Normally code is written like:

```
1 Service → calls → DoclingClient()
```

</> Python

But with IoC:

```
1 Domain defines "I need a DocumentExtractor"
2 Framework injects the Docling adapter into the domain
```

</> Python

### Where IoC appears in your system

Your diagram shows IoC boundary:

```
1 Core Domain <-- depends on Ports
2 Adapters <-- implement Ports
```

</> Python

Domain does **NOT import adapters**.

Adapters import the **domain port**.

This flips dependencies upside down (inverted!).

## Benefit Example

In your ingestion pipeline:

- Pipeline calls `DocumentExtractionPort`
- At startup, IoC injects:
  - `DoclingExtractorAdapter` in production
  - `FakeExtractor` in tests

Meaning your domain is never tied to concrete services.

## Benefits

- Super testable
  - Test ingestion without **running Docling server**
- Maintainable
  - Replace parts without touching **domain logic**
- Follows SOLID principles
  - Especially the "Dependency Inversion Principle"

## Downsides

- Hard to learn
- Requires dependency injection patterns
- More layers = more complexity for small teams

# 4. Pipes & Filters (Pipeline Design)

## Where it appears

In your backend:

```
1 Architecture Framework: Pipes & Filters
```

</> Python

Every step is a **filter**:

```
1 Extract → Chunk → Embed → Store → Index
```

</> Python

The pipeline itself is a **pipe**.

## What does it mean?

Each step:

- does **ONE** job
- has a **simple input → output**
- **doesn't know other steps**

Like Unix pipes:

```
1 cat document | extract | chunk | embed | store
```

</> BAT

Example from your system

Extraction Step:

</> Python

```
1 async def __call__(context):
2 context.chunks = extractor.extract(context.file_bytes)
3
```

Chunking Step:

</> Python

```
1 async def __call__(context):
2 context.final_chunks = chunker.chunk(context.chunks)
3
```

Each step is replaceable.

Benefits

- Flexible workflows
  - Want OCR before extraction? Add a step.
  - Want translation? Add a step.
- Extremely modular
  - Each step is testable in isolation.
- Parallelization possible

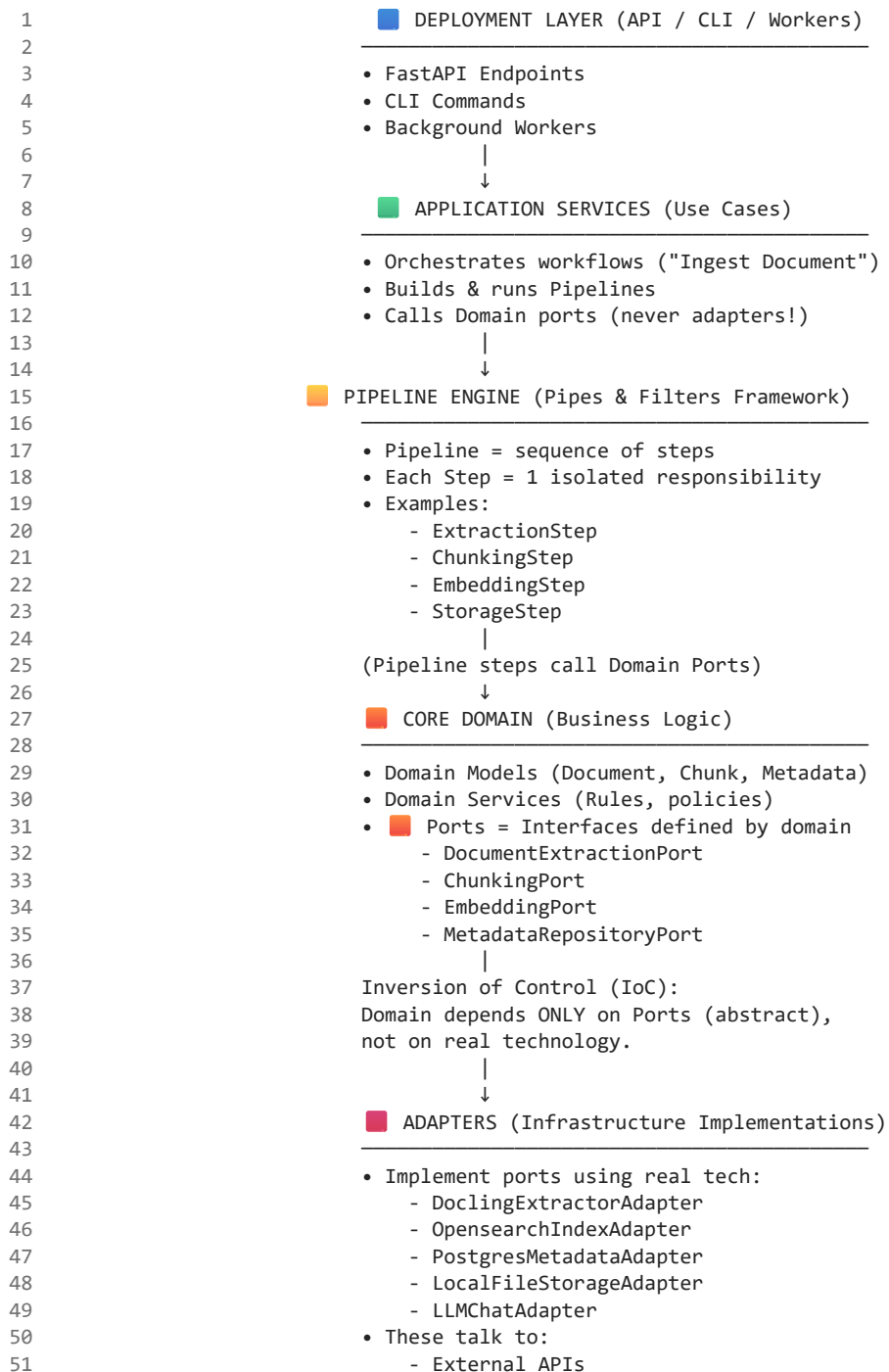
Downsides

- A lot of context-passing
- Harder debugging across many small steps
- Pipeline becomes complex if misused

|   | ≡ Paradigm       | ≡ Meaning                                          | ≡ Where Used                 | ≡ Why It's Good                 |
|---|------------------|----------------------------------------------------|------------------------------|---------------------------------|
| 1 | DDD              | Domain rules in the center                         | /domain                      | Clean, stable, business-focused |
| 2 | Ports & Adapters | Abstractions vs implementations                    | /domain/ports ,<br>/adapters | Replace Docling easily          |
| 3 | IoC              | Domain depends on ports, adapters depend on domain | Everywhere                   | Decoupled, testable system      |

|   |                      |                        |                                                     |                                           |
|---|----------------------|------------------------|-----------------------------------------------------|-------------------------------------------|
| 4 | Pipes & Filters      | Modular pipeline steps | <code>/pipelines/*</code>                           | Flexible ingestion / RAG                  |
| 5 | Application Services | Orchestrate use cases  | <code>/application/service</code><br><code>s</code> | Clean separation between logic & delivery |
| 6 | Deployment Layer     | FastAPI, CLI           | <code>/api/</code>                                  | Interface to the real world               |

## Architecture Diagram



```

52 - Databases
53 - File systems
54 - LLM models
55 ↓
56  INFRASTRUCTURE (External Services)
57 -----
58
59 • Docling HTTP API
60 • Opensearch Server
61 • PostgreSQL
62 • Vector Store
63 • LLM Server / vLLM
64
65
66

```

Mermaid ▾

```

1 flowchart TB
2
3 %% DEPLOYMENT LAYER
4 subgraph Deployment_Layer
5 API[FastAPI Endpoints]
6 CLI[CLI Commands]
7 Worker[Background Workers]
8 end
9
10 %% APPLICATION LAYER
11 subgraph Application_Layer
12 AppService[DocumentIngestionService]
13 end
14
15 %% PIPELINE ENGINE
16 subgraph Pipeline_Engine
17 ExtractStep[Extraction Step]
18 ChunkStep[Chunking Step]
19 EmbedStep[Embedding Step]
20 StoreStep[Storage Step]
21 end
22
23 %% CORE DOMAIN
24 subgraph Core_Domain
25 DomainModels[Domain Models]
26 DomainServices[Domain Services]
27
28 subgraph Domain_Ports
29 ExtractionPort[DocumentExtractionPort]
30 ChunkingPort[ChunkingPort]
31 EmbeddingPort[EmbeddingPort]
32 MetadataPort[MetadataRepositoryPort]
33 end
34 end
35
36 %% ADAPTERS
37 subgraph Adapters_Layer
38 DoclingAdapter[DoclingExtractorAdapter]
39 OpenSearchAdapter[OpenSearchIndexAdapter]
40 PostgresAdapter[PostgresMetadataAdapter]
41 LLMAAdapter[LLMChatAdapter]
42 end
43
44 %% INFRASTRUCTURE
45 subgraph Infrastructure_Layer
46 DoclingAPI[Docling HTTP API]
47 SearchServer[OpenSearch Server]
48 PostgresDB[PostgreSQL Database]
49 LLMServer[LLM Server]

```



```

50 end
51
52
53 %% DATA FLOW
54 API --> AppService
55 CLI --> AppService
56 Worker --> AppService
57
58 AppService --> ExtractStep
59 ExtractStep --> ChunkStep
60 ChunkStep --> EmbedStep
61 EmbedStep --> StoreStep
62
63 %% PIPELINE STEPS CALL DOMAIN PORTS
64 ExtractStep --> ExtractionPort
65 ChunkStep --> ChunkingPort
66 EmbedStep --> EmbeddingPort
67 StoreStep --> MetadataPort
68
69 %% IOC: DOMAIN PORTS CONNECT TO ADAPTERS
70 ExtractionPort --> DoclingAdapter
71 ChunkingPort --> DoclingAdapter
72 EmbeddingPort --> LLMAdapter
73 MetadataPort --> PostgresAdapter
74 MetadataPort --> OpenSearchAdapter
75
76 %% ADAPTERS CALL EXTERNAL INFRASTRUCTURE
77 DoclingAdapter --> DoclingAPI
78 OpenSearchAdapter --> SearchServer
79 PostgresAdapter --> PostgresDB
80 LLMAdapter --> LLMServer
81

```

